CRC Package Interface Specification

Current API design

Table of contents

1 Overview

The package is centred around a single high-level function:

```
crc_fit(
  data,
  captures,
  capture_formula,
  outcome = NULL,
  outcome_formula = NULL,
  outcome_dist = c("ztnegbin", "ztpois"),
```

```
  misclass = NULL,
  latent_classes = 1L,
  control = NULL
)
```

The current implementation focuses on **validation and parsing**. Model fitting (EM algorithm) will be implemented later.

2 Main arguments

Argument	Description
`data`	One row per observed unit.
`captures`	One-sided formula defining capture indicators.
`capture_formula`	Log-linear capture model. `.latent` denotes the latent class.
`outcome`	`NULL`, a single outcome variable, or `c(lower=..., upper=...)`.
`outcome_formula`	Mean model for the outcome distribution.
`outcome_dist`	Currently `"ztnegbin"` or `"ztpois"`.
`misclass`	Misclassification specification for categorical variables.
`latent_classes`	Number of latent classes.
`control`	Numerical control options (reserved for future use).

3 Current workflow

```
crc_fit()

   match.arg(outcome_dist)
   validate_crc_input()
   parse_captures()
   parse_outcome()
   parse_misclass()
   parse_capture_formula()
   parse_outcome_formula()
```

At this stage the function returns an **unfitted model object** containing parsed specifications.

4 Parser responsibilities

4.1 `parse_captures()`

- validates the capture formula;
- verifies binary capture indicators;
- checks missing values;
- checks that every unit is observed by at least one source;
- returns a `crc_captures` object.

4.2 `parse_outcome()`

Supports:

- `NULL`;
- one exact outcome column;
- interval/right-censored outcomes via `lower`/`upper`.

Returns a standardized table with:

- `lower`
- `upper`
- `status`

4.3 `parse_capture_formula()`

Validates:

- one-sided formula;
- intercept;
- untransformed variables only;
- presence of all capture main effects;
- hierarchical interactions;
- `.latent`;
- allowed interactions:
  - capture × capture,
  - capture × `.latent`.

Returns a `crc_capture_formula` object.

4.4 `parse_outcome_formula()`

Validates:

- one-sided formula;
- intercept;
- categorical predictors;
- hierarchical interactions;
- optional use of `.latent`.

Returns a `crc_outcome_formula` object.

4.5 `parse_misclass()`

Expected structure:

```
misclass = list(
  occupation = list(
    matrix = Pi_occ,
    true_if = ~ source_1
  ),
  contract = list(
    matrix = Pi_contract
  )
)
```

Responsibilities:

- validates confusion matrices;
- validates category levels;
- validates `true_if`;
- constructs logical vectors identifying observations with known true category;
- returns a `crc_misclass` object.

5 Returned object

Currently `crc_fit()` returns an object of classes

```
c("crcfit_unfitted", "crcfit")
```

containing:

- parsed capture specification,
- parsed outcome specification,
- parsed capture model,
- parsed outcome model,
- parsed misclassification specification,
- model metadata.

6 Next implementation steps

1. Build internal model matrices.
2. Initialize parameters.
3. Implement the EM algorithm.
4. Compute standard errors.
5. Add `summary()`, `print()`, and `predict()` methods.

CRC Package Interface Specification

Model-matrix and initialization implementation addendum

1 Model-matrix infrastructure

The package now converts parsed model specifications into complete capture-cell grids and aligned observation-level states. The infrastructure includes the capture and outcome design matrices, misclassification contributions, and a direct mapping from each observation state to its capture cell. Construction of these components is integrated into `crc_fit()`, although model fitting has not yet been implemented.

1.1 Capture-model matrix

`build_capture_matrix()` constructs the Cartesian product of all capture histories, categorical levels used by the capture model, and latent classes. For misclassified variables, true-category levels and their ordering are obtained from the corresponding confusion matrices. The function returns:

- `cells`, the complete capture-model cell grid;
- `matrix`, the design matrix generated from the parsed terms;
- `unobserved`, an indicator of all-zero capture histories.

With J capture sources, K latent classes, and categorical variables with $C_1, \ldots, C_p$ levels, the grid contains $2^J K \prod_{q=1}^{p} C_q$ rows.

1.2 Initial observation states

`build_observation_states()` repeats every observed unit once for each latent class. It retains the variables needed by the capture model, outcome model, and misclassification mechanisms. It also keeps aligned copies of the standardized outcome and observed values of misclassified variables. An observation identifier maps every expanded row back to its original unit. Only variables used by the capture or outcome model are retained in the state data. Recorded values of variables subject to misclassification are stored separately for evaluation of their confusion-matrix contributions.

1.3 Misclassified-variable expansion

`expand_misclass_states()` expands the current states over every possible true level of one misclassified variable. Each expanded row receives the contribution

$$\Pr(\widetilde{G}_j = h \mid G_j = g)$$

from its confusion matrix. When `true_if` applies, the observed category receives probability one and all other candidate categories receive probability zero.

For several misclassified variables, separate confusion matrices are used and their contributions are multiplied. This corresponds to conditional independence of the misclassification mechanisms given their respective true categories. Dependent mechanisms may be supported in a future extension. These contributions are likelihood terms, not posterior probabilities, and must not be normalized at this stage.

`expand_all_misclass_states()` applies this expansion successively to every specified variable. When no misclassification mechanism is supplied, it assigns a contribution of one to every latent state.

1.4 Outcome-model matrix

`build_outcome_matrix()` applies the parsed outcome terms to the fully expanded state data. Its rows therefore remain aligned with the state data, standardized outcomes, observation identifiers, and accumulated misclassification contributions. It returns `NULL` when no outcome model is specified.

Every predictor in `outcome_formula` is required to occur in `capture_formula`. This ensures that the predictor is represented in the complete capture-cell grid and is consequently available for prediction in the all-zero cells. The formulas may contain different terms; only the set of variables is constrained.

1.5 Capture-cell indices

`build_capture_index()` returns one integer index for every expanded observation state. Matching uses the capture indicators, categorical variables in the capture model, and latent class. If the ith value is r, row i of the state data uses row r of the capture-cell grid and capture design matrix. The vector avoids repeated joins during future EM iterations. Construction fails if any state cannot be matched to the grid.

1.6 Unified model-matrix object

`build_model_matrices()` coordinates the complete construction sequence and returns an object of class `crc_model_matrices` with:

- `capture`, containing the cell grid, capture design matrix, and all-zero-cell indicator;
- `states`, containing expanded state data and aligned metadata;
- `outcome_matrix`, containing the outcome design matrix or `NULL`;
- `capture_index`, linking state rows to capture-cell rows.

`crc_fit()` constructs this object after parsing and compatibility validation and stores it as `model_matrices`. The returned object remains in class `crcfit_unfitted` and has `fitted = FALSE`.

1.7 Cross-component validation

Compatibility checks now reject outcome predictors absent from the capture model. They also reject variables specified in `misclass` that are used by neither the capture nor the outcome model. This prevents redundant state expansion and makes the population-cell interpretation explicit.

2 Initialization

`initialize_crc()` creates normalized initial weights for the expanded observation states and expected counts for the complete capture-cell grid. For multiple latent classes, independent gamma draws produce a symmetric Dirichlet allocation within each observation and candidate true-category configuration. These allocations are multiplied by the misclassification contributions and normalized within each observed unit. The concentration is set by `control$init_alpha` and defaults to 20.

State weights are aggregated through the precomputed capture-cell indices. Observed cells receive the corresponding weighted counts, while every all-zero capture cell receives an initial expected count of one. The resulting `crc_initialization` object stores the state weights, cell counts, and Dirichlet concentration and is retained by `crc_fit()`.

3 Automated tests

The model-matrix and initialization infrastructure is covered by automated tests implemented with the `tinytest` package. Separate test files cover model matrices, misclassification expansion, cross-component compatibility, and initialization.

The tests verify capture-grid and design-matrix dimensions, row alignment, factor-level order, all-zero capture cells, and capture-cell indices. They also cover multiple misclassified variables, multiplication of confusion-matrix contributions, and `true_if` restrictions. Models without an outcome or misclassification mechanism and both new compatibility errors are included. All tests exercise the implemented behavior through `crc_fit()` and pass against the built and installed package.

Initialization tests additionally verify state-weight dimensions, finiteness, non-negativity, within-observation normalization, capture-cell aggregation, the default and custom concentration values, reproducibility under `set.seed()`, the single-class case, and invalid-control handling.

4 Required next steps

The model-matrix and initialization infrastructure and their automated tests are complete at the design level. Development should continue in the following order.

1. Implement the E-step by combining capture-cell expectations, misclassification contributions, and exact, censored, or missing outcome likelihood contributions. Normalize the resulting weights within each observed unit.
2. Implement separate M-step updates for the capture and outcome models, using the posterior state weights and the existing alignment indices.
3. Add convergence controls, numerical safeguards, and diagnostics before exposing fitted estimates through user-facing methods.

EM updates must follow the methodology in the accompanying paper and simulation implementation. The existing automated tests establish the initialization, matrix, and alignment invariants on which those iterative components depend.